

Universidad Autónoma de Yucatán
Facultad de Matemáticas
Asignatura: Construcción de Software

Prueba de Desempeño 1
2da Oportunidad

Unidad 1

Estudiante: _____ **Fecha:** _____

Valor: 8 pts

Forma de trabajo: Individual

Instrucciones generales

Para cada uno de los 3 ejercicios debes realizar lo siguiente:

1. **Analizar** el código fuente proporcionado.
2. **Listar** de forma escrita como comentario, todos los errores encontrados, según los temas estudiados en la unidad 1:
 - a) *Técnicas de construcción de variables y tipos de datos fundamentales*
 - b) *Técnicas de organización de sentencias.*
 - c) *Técnicas de construcción de estructuras de control de flujo.*
 - d) *Técnicas de construcción de procedimientos.*
 - e) *Técnicas de construcción de clases*
 - f) *Técnicas de documentación de código*
3. **Reestructurar** el código aplicando buenas prácticas de construcción de software (nombres significativos, convenciones Java, bajo acoplamiento, alta cohesión, manejo adecuado de excepciones, reducción de complejidad ciclomática, principio de responsabilidad única, modularidad etc.).
4. **Entregar** el código corregido escrito manteniendo la funcionalidad original agregando la función *main* para que pueda compilar y ejecutarse con una prueba de caja negra..

Importante: No se aceptan soluciones que solo renombren variables sin corregir los problemas estructurales de fondo.

Ejemplo

Funcionalidad:

Este programa clasifica un número (del 1 al 7) como "Día laboral" o "Fin de semana", y además verifica si un turno de trabajo corresponde al turno de la mañana comparando cadenas de texto.

Código original

```
1  public class ClasificadorDia {
2      public static String clasificar(int dia) {
3          String tipo = "";
4          switch (dia) {
5              case 1:
6              case 2:
7              case 3:
8              case 4:
9              case 5:
10             tipo = "Día laboral";
11             case 6:
12             case 7:
13             tipo = "Fin de semana";
14             break;
15             default:
16             tipo = "Día inválido";
17         }
18         return tipo;
19     }
20
21     public static boolean esTurnoManana(String turno) {
22         String entrada = new String("Mañana");
23         return turno == entrada;
24     }
25
26     public static void main(String[] args) {
27         System.out.println(clasificar(3));
28         System.out.println(esTurnoManana("Mañana"));
29     }
30 }
```

Análisis

```
1  // ERROR: sin Javadoc de clase.
2  // ERROR: sin Javadoc (propósito, @param, @return).
3  // ERROR: nombre genérico ("clasificar" no dice qué clasifica).
4  public class ClasificadorDia {
5      public static String clasificar(int dia) {
6          // ERROR: inicialización innecesaria con cadena vacía.
7          String tipo = "";
8          switch (dia) {
9              // ERROR: números mágicos (1 a 7) sin significado; cadenas literales dispersas.
10             case 1:
11             case 2:
12             case 3:
13             case 4:
14             case 5:
15             // ERROR: falta "break"; hay "fall-through" y la ejecución
16             // continúa en el case 6/7, sobrescribiendo con "Fin de semana".
17             // clasificar(3) devuelve "Fin de semana" en lugar de "Día laboral".
18             tipo = "Día laboral";
19             case 6:
20             case 7:
21             tipo = "Fin de semana";
22             break;
23             default:
24             // ERROR: el default devuelve un texto como si fuera un tipo válido; para una
25             // entrada inválida es mejor lanzar una excepción.
26             tipo = "Día inválido";
27         }
28         // ERROR: variable intermedia innecesaria; se puede retornar directo.
29         return tipo;
30     }
31
32     // ERROR: sin Javadoc.
33     public static boolean esTurnoManana(String turno) {
34         // ERROR: no valida que "turno" sea null.
35         // ERROR: "new String(...)" crea un objeto innecesario y fuerza una referencia distinta
36         // al literal del pool; mala práctica.
37         // ERROR: literal "Mañana" sin constante.
38         String entrada = new String("Mañana");
39         // ERROR: "==" compara referencias, no contenido; devuelve false
40         // aunque el texto sea igual. Debe usarse equals().
41         return turno == entrada;
42     }
43
44     // ERROR: sin Javadoc.
45     public static void main(String[] args) {
46         // ERROR: imprime sin verificar el resultado esperado
47         System.out.println(clasificar(3));
48         System.out.println(esTurnoManana("Mañana"));
49     }
50 }
51
```

Propuesta de solución

```
1  /**
2   * Clasifica los días de la semana y valida turnos de trabajo.
3   */
4  public class ClasificadorDia {
5
6      /** Primer día de la semana (lunes). */
7      private static final int PRIMER_DIA = 1;
8
9      /** Último día laboral (viernes). */
10     private static final int ULTIMO_DIA_LABORAL = 5;
11
12     /** Último día de la semana (domingo). */
13     private static final int ULTIMO_DIA = 7;
14
15     /** Texto para los días de lunes a viernes. */
16     private static final String DIA_LABORAL = "Día laboral";
17
18     /** Texto para sábado y domingo. */
19     private static final String FIN_DE_SEMANA = "Fin de semana";
20
21     /** Nombre del turno de la mañana. */
22     private static final String TURNO_MANANA = "Mañana";
23
24     /**
25      * Clasifica un día de la semana como laboral o fin de semana.
26      *
27      * @param dia número del día (1 = lunes ... 7 = domingo)
28      * @return "Día laboral" para 1 a 5, "Fin de semana" para 6 y 7
29      * @throws IllegalArgumentException si el día no está entre 1 y 7
30      */
31     public static String clasificarDiaSemana(int dia) {
32         if (dia < PRIMER_DIA || dia > ULTIMO_DIA) {
33             throw new IllegalArgumentException("Día inválido: " + dia);
34         }
35         return dia <= ULTIMO_DIA_LABORAL ? DIA_LABORAL : FIN_DE_SEMANA;
36     }
37
38     /**
39      * Indica si un turno corresponde a la mañana.
40      *
41      * @param turno nombre del turno (puede ser null)
42      * @return true si el turno es "Mañana", false en caso contrario
43      */
44     public static boolean esTurnoManana(String turno) {
45         return TURNO_MANANA.equals(turno);
46     }
47
48     /**
49      * Punto de entrada del programa.
50      *
51      * @param args argumentos de línea de comandos (no se utilizan)
52      */
53     public static void main(String[] args) {
54         System.out.println(clasificarDiaSemana(3));
55         System.out.println(clasificarDiaSemana(7));
56         System.out.println(esTurnoManana("Mañana"));
57         System.out.println(esTurnoManana("Tarde"));
58     }
59 }
```

Ejercicio 1 (3 pts)

Funcionalidad:

Este programa recorre un arreglo de enteros sumando sus valores y omitiendo (según su propio mensaje en consola) los valores negativos.

Código original

```
1  public class Procesador {
2      public static void procesar(int[] datos) {
3          int i = 0;
4          int suma = 0;
5          while (i < datos.length) {
6              suma += datos[i];
7              if (datos[i] < 0) {
8                  System.out.println("Valor negativo encontrado, se omite");
9                  continue;
10             }
11             i++;
12         }
13         System.out.println("Suma total: " + suma);
14     }
15
16     public static void main(String[] args) {
17         int[] datos = {5, 10, -3, 8};
18         procesar(datos);
19     }
20 }
```

Análisis

Propuesta de solución

Ejercicio 2 (2 pts)

Funcionalidad:

Este programa recibe una lista de nombres de clientes y elimina de la lista a aquellos cuyo nombre coincide con un nombre dado como "inactivo".

Código original

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class GestorClientes {
5      public static void eliminarInactivos(List<String> clientes, String inactivo) {
6          for (String cliente : clientes) {
7              if (cliente == inactivo) {
8                  clientes.remove(cliente);
9              }
10         }
11     }
12
13     public static void main(String[] args) {
14         List<String> clientes = new ArrayList<>();
15         clientes.add("Juan");
16         clientes.add("Pedro");
17         clientes.add(new String("Pedro"));
18         eliminarInactivos(clientes, "Pedro");
19         System.out.println(clientes);
20     }
21 }
```

Análisis

Propuesta de solución

Ejercicio 3 (3 pts)

Funcionalidad:

Este programa modela figuras geométricas (un rectángulo y un triángulo) y calcula el área de cada una. La clase Procesador recibe una figura y, dependiendo de si es o no un triángulo, imprime su área en consola. Es decir, simula un sistema simple de procesamiento de formas geométricas donde cada figura sabe calcular su propia área.

Código original

```
1  public class Figura {
2      public double base;
3      public double altura;
4
5      public double calcularArea() {
6          return base * altura;
7      }
8  }
9
10 public class Triangulo extends Figura {
11     public double calcularArea() {
12         return (base * altura) / 2;
13     }
14 }
15
16 public class Procesador {
17     public void imprimirArea(Figura figura) {
18         if (figura instanceof Triangulo) {
19             Triangulo t = (Triangulo) figura;
20             System.out.println("Área del triángulo: " + t.calcularArea());
21         } else {
22             System.out.println("Área: " + figura.calcularArea());
23         }
24     }
25
26     public static void main(String[] args) {
27         Procesador p = new Procesador();
28         Figura rectangulo = new Figura();
29         rectangulo.base = 4;
30         rectangulo.altura = 5;
31
32         Triangulo triangulo = new Triangulo();
33         triangulo.base = 4;
34         triangulo.altura = 5;
35
36         p.imprimirArea(rectangulo);
37         p.imprimirArea(triangulo);
38     }
39 }
```

Análisis

Propuesta de solución